Fast, Error-Resilient Intent Recognition for Offline Security Copilots: A Guarded LLM Pipeline

P. M. T. L. Karunarathna
Department of Computer System Engineering
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
it22249852@my.sliit.lk

Abstract—Large language models (LLMs) are increasingly proposed as copilots for penetration testing and defensive security operations. However, LLMs hallucinate fabricated CVEs, malformed commands, and parameter values, and they can be diverted by adversarial prompt engineering, which makes their output unusable in operational security tooling. This paper presents a guarded, retrieval-free intent-recognition engine that wraps an offline, CPU-only attacker-guided LLM (Qwen or Gemma; exact model tags in Section VI) with a seven-stage normalizing pipeline, three-layer adversarial detection, role-based access control, deterministic schema and network validation, and a semantic self-consistency gate. We contribute a purpose-built 900-record data set (400 well-formed and 500 flawed prompts across eight categories) and an ablation methodology that isolates each defense layer. Across four experiments on a single consumer laptop, the guarded pipeline raises intent-recognition accuracy from 69–78% to 82–86%, raises flaw-catch rate from 0% to 72% (Gemma) and 64% (Qwen) on the stratified holdout and to 84% on the 900-record ablation set, and cuts false positives from 31–32% to 14–15%; the largest single layer (hallucination validation) contributes up to 22 points of catch rate. All measurements are single-run point estimates from one backend pass per configuration, without confidence intervals or seed sweeps; the resulting uncertainty is discussed in the Limitations section. We discuss failure modes unique to offline LLM backends and analyze the residual threats.

Index Terms—intent recognition, LLM guardrails, hallucination detection, prompt injection, RBAC

I. INTRODUCTION

Security operations today are bottlenecked by expertise. Penetration testers, security analysts, and blue teams must map free-form intent onto tools, commands, and targets while complying with authorization boundaries. Recent work has shown that large language models (LLMs) can act as AI sparring partners for this work, producing realistic attack plans and even executing exploitation steps in controlled environments [1], [2]. Demonstrations of indirect prompt injection and dual-use abuse, however, show that the same model that helps a defender can be hijacked by a user or by text retrieved from a web page [3]–[5].

An intent-recognition engine sits between the operator and the security toolchain: it classifies what the operator wants — run a port scan, check a CVE, brute-force a password — and emits a machine-consumable structure. If that engine is powered by an LLM, three classes of failure emerge. First, hallucination: the model fabricates a CVE identifier, a parameter, or a claim, and the downstream tool acts on non-existent data [7], [10]. Second, adversarial deflection: a prompt is worded so that the model abandons the classification task and emits attacker-chosen text [3], [4]. Third, authorization leakage: unrestricted models answer out-of-policy request categories (exploitation, password attacks) for users who lack permission.

This paper addresses the question: can a small, offline, unattended LLM be made reliable enough to drive an operational security copilot? We answer with NeuroShell IRE (Intent Recognition Engine), a guarded pipeline in which a 7 B-parameter model is treated as an unreliable proposition generator whose output is validated deterministically. The architecture (Section V) combines schema and regex validation, a network validator that checks targets against authorization policy, a scope guard, a three-layer adversarial detector, role-based access control (RBAC), a semantic cache, and a self-consistency gate built on semantic-clustering agreement. Four experiments (Section VIII) on a consumer laptop without a GPU measure accuracy, flaw catch, and ablation impact of every layer, and identify the largest contributors to safety. The dataset and evaluation harness are versioned with the project source and reproducible from a single evaluation manifest.

The contributions of this work are: (i) a seven-stage, five-middleware architecture (S6B is the network sub-validator of S6) that subjects an offline LLM's output to deterministic, auditable validation before use; (ii) a purpose-built 900-record data set spanning eight flaw categories; (iii) a per-layer ablation methodology and results quantifying each defense's contribution; and (iv) a quantitative comparison of two LLM backends (Gemma and Qwen) under the same guardrail stack.

II. RELATED WORK

LLMs for security tooling. Happe and Cito [2] proposed LLMs as sparring partners for penetration testing and demonstrated a closed-loop system that drove a vulnerable virtual machine to root. Deng et al. [1] built PEN-

TESTGPT, an LLM-centric assistant that decomposes penetration-testing tasks across a reasoning, generation, and guidance chain. Both works focus on the generation side — producing plausible next actions — and treat output correctness informally; neither validates or authorizes the output before execution.

Attacks on LLM-integrated applications. Greshake et al. [3] introduced indirect prompt injection, where an attacker plants instructions in data that the model later ingests, and demonstrated compromise of real-world systems. Liu et al. [4] formalized prompt-injection attacks and systematically benchmarked ten defenses, finding that prevention-based defenses were weak and detection-based defenses incomplete. Kang et al. [5] showed that dual-use attacks inspired by classic computer security bypass in-the-wild LLM content defenses with near-perfect success. The OWASP Top 10 for LLM Applications [6] formally catalogues these risks — prompt injection (LLM01), sensitive-information disclosure (LLM02), and others — as first-class application concerns.

Hallucination detection. Ji et al. [7] survey hallucination across natural-language generation, distinguishing intrinsic from extrinsic hallucination. Sampling-based methods detect failures without external knowledge: Manakul et al. [8] proposed SelfCheckGPT, which scores the consistency of sampled responses; Kuhn et al. [9] framed answer agreement via semantic uncertainty, and Farquhar et al. [10] used semantic entropy over meaning clusters to detect confabulations in free-form text. These methods assume interactive or repeated generation; our contribution applies the disagreement intuition to a single guarded classification with a hard latency budget, which we argue is the more realistic constraint in tool-in-the-loop security products.

Retrieval and embeddings. Retrieval-augmented generation (RAG) [12] grounds answers in external corpora, and sentence-level embeddings trained with bi-encoders (Sentence-BERT) [13] provide the semantic distance signals used by our self-consistency gate and semantic cache. Unlike RAG, NeuroShell IRE deliberately performs no retrieval at inference time, keeping the offline, single-device deployment boundary free of new attack surface (retrieved data is itself a prompt-injection vector [3]).

III. RESEARCH GAP

Two gaps motivate this work. First, the LLM-security copilot systems examined above validate their output informally, if at all: outputs are plausible prose consumed by humans. When a copilot's output is fed to an automated toolchain — the natural integration point for intent recognition — hallucinated CVEs, malformed JSON, and injected text become machine-dispatched actions, not display text. The failure semantics change and the defense requirement becomes correctness, not plausibility. Second, the security literature evaluates defenses on cloud-hosted models with interactive budgets; we are not aware of prior work reporting a quantitative, per-layer ablation of a guardrail stack for a small offline model running unattended in the field, where a retry loop or a verifier LLM doubles a multi-second CPU inference cost. We therefore ask three concrete research questions:

- RQ1 How much does a stacked validation pipeline improve structured intent-recognition accuracy and flaw catching over raw LLM output?
- RQ2 Which individual guardrail layer contributes the most to catch rate, and which layers interact destructively?
- RQ3 Do the guardrails generalize across two different offline LLM backends?

IV. RESEARCH OBJECTIVES

To answer the research questions we pursue four objectives:

1) Build a guarded intent-recognition engine that consumes free-text security requests and emits strictly-schema-validated, authorized intent structures with an average end-to-end latency compatible with single-user offline use.
2) Construct a 900-record data set covering 400 well-formed prompts and 500 deliberately flawed prompts across eight flaw categories (adversarial injection, out-of-scope, ambiguous, contradictory, fabricated parameter, fabricated CVE, type mismatch, and RBAC violation).
3) Establish an ablation protocol in which each guardrail is independently disabled while all others remain active, enabling an honest causal estimate of each layer's contribution.
4) Compare two offline LLM backends (Gemma and Qwen) under the identical guardrail stack to measure backend-agnostic coverage.

V. NEUROSHELL IRE ARCHITECTURE

NeuroShell IRE is a Python service (FastAPI) deployed as a local daemon that exposes an HTTP API for downstream security tools. It runs entirely on the operator's laptop; no request leaves the device. Fig. 1 shows the pipeline. A core of seven serialized stages (S1–S7, where S6B is a network sub-validator of S6 rather than an eighth stage) transforms the raw utterance into a validated intent, while five middleware components enforce non-negotiable safety properties independently of the core.

Core stages. S1 normalizes casing, whitespace, and unicode. S2 resolves aliases (e.g., "nmap -sS" [17] $\rightarrow$ SCAN) against a catalog of 26 sub-intents spanning nine top-level intents (SCAN, EXPLOITATION, PASSWORD_ATTACK, DIRECTORY_BRUTEFORCE, CVELOOKUP, RECON, PHISHING, NETWORKDEFENSE, MISC). S3 asks the LLM to emit a fixed JSON schema at temperature 0.4. S4 parses the response and fails fast on malformed JSON. S5 validates against the schema;

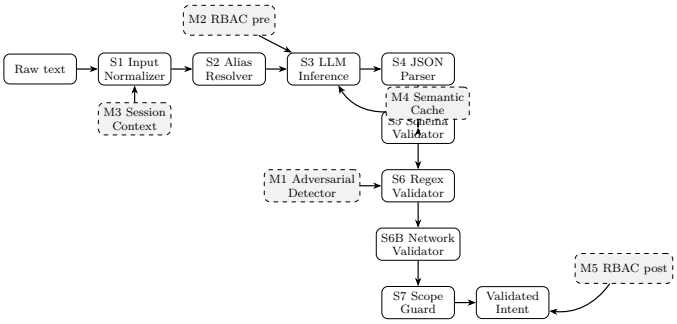


Fig. 1. Guarded intent-recognition pipeline. Solid boxes run on every request; dashed middleware enforce security invariants and shortcut the core where possible.

S6 applies regex rules (host syntax, CIDR, port ranges). S6B, the network validator, resolves the target and checks it against the authorization policy and RFC 1918 private ranges [14]. S7, the scope guard, rejects requests whose target or technique falls outside the operator's declared scope. Only output that survives S5–S7 is returned; otherwise the stage-specific guard emits a classified error.

Middleware. M1 is the three-layer adversarial detector: a lexical layer assigns suspicion to prompt tokens (weights 0.8–1.0), a structural layer scores sentence topology and imperative-but-unclassifiable phrasing (0.3–0.9), and a scoring layer fuses both cumulatively; a cumulative score above 0.7 flags the request, and 0.95 auto-blocks without inference. M2/RBAC pre checks the caller's role before any inference is paid for; a viewer role fast-fails before S3. M3 maintains session context (user, role, target history). M4 is a semantic cache keyed on normalized embedding distance (Sentence-BERT [13]): identical intents short-circuit the LLM entirely. M5 re-validates authorization on the final intent, so a single bypassed guard is not by itself sufficient to manufacture an authorized action.

RBAC model. Four roles (viewer, analyst, operator, admin) gate the nine intents. The three sensitive intents — EXPLOITATION, PASSWORD_ATTACK, DIRECTORY_BRUTEFORCE — require at least operator; EXPLOITATION covers offensive tooling drivers such as the Metasploit Framework [18]. The default role is analyst; an unknown caller is downgraded to viewer. Enforcement occurs both pre-inference (M2, before cost is incurred) and post-validation (M5, on the final intent).

Deployment properties. The service runs headless, retrieves nothing at inference time (no RAG), and holds no secrets other than the model weights. Because there is no retrieval and no free-form output is ever rendered to a downstream shell, the two dominant remote attack classes — indirect prompt injection via retrieved data [3] and tool-injection via unfiltered text — are structurally reduced to the narrower problem of classifying the single user utterance.

Threat model. Assets are the operator's security toolchain and system state. The trust boundary is the single device: no request leaves it, nothing is retrieved, and no output is rendered. The attacker controls the user utterance but not the device, prompts, policies, or model weights. Under this boundary the evaluated attack surfaces are direct adversarial utterances (handled by M1 and the RBAC matrix; 95.6% of adversarial records caught) and authorization escalation (M2/M5). Indirect prompt injection via retrieved content and tool-injection via rendered text are structurally out of scope because neither channel exists; physical compromise of the device and backdoors embedded in the model weights are out of scope.

Formal model. Let $\mathcal{X}$ be the utterance space, $\mathcal{Y}$ the nine-intent space, $\mathcal{C}$ the eight flaw classes, and $\mathcal{R}$ the role set. The decoder is a stochastic map $f : \mathcal{X} \to \mathcal{Z}$ producing observation $z \sim f(x)$; the deterministic guards are boolean predicates $g_5(z)$, $g_6(z)$, $g_{6B}(z, x)$, $g_7(x)$ over S5–S7, and the RBAC check $b : \mathcal{R} \times \mathcal{Y} \to \{0, 1\}$. The guarded outcome is the decision

$$\Phi(x) = \begin{cases} y \in \mathcal{Y} \text{ accepted}, & G(z,x) \wedge b(\rho, y) \wedge \neg\, m_1(x), \\ e \in \mathcal{C} \text{ blocked}, & \text{otherwise}, \end{cases} \tag{1}$$

with $G(z, x) = g_5 \wedge g_6 \wedge g_{6B} \wedge g_7$ and ρ the caller role. The safety invariant the stack enforces is that every accepted output is schema-valid, network-authorized, in-scope, role-permitted, and non-adversarial:

$$\text{Accepted}(\Phi(x)) \;\Rightarrow\; \bigwedge\nolimits_{k \in \{S5, S6, S6B, S7, \text{RBAC}, M1\}} P_k(x). \tag{2}$$

The pipeline value (Exp1) is then precisely the gap in $\Pr[\bigwedge_k P_k]$ between raw f and guarded Φ; and a layer ablation (Exp3) estimates Pr[Accepted] with each predicate P_ℓ removed — the ΔCatch column.

VI. METHODOLOGY

Setup. All runs executed on an ASUS VivoBook X513EA (Intel Core i5-1135G7, 4 cores/8 threads, 24 GB DDR4, Windows 11) with Ollama in CPU-only mode. Backends are qwen2.5-coder:7b (4.7 GB) and gemma4:e4b (9.6 GB on disk), the build to which the gemma4:latest alias resolved at evaluation time. The exact Ollama version and per-model image digests as fetched are pinned in the evaluation manifest, which is the authoritative anchor for reproducing this study. No GPU is used; this reproduces the field deployment.

Dataset. Table I summarizes the 900-record data set (D): 400 well-formed prompts (the D^+ set) and 500 flawed prompts (D^-) across eight categories, each with a gold intent label and a gold flaw label. Category sizes are deliberately non-uniform (24–100 records per class), reflecting per-category authoring feasibility rather than experimental design; per-class catch rates are therefore reported against each class's own size, and cross-category comparisons should be read with the differing denominators in mind.

TABLE I
Data set composition (N=900). D^+ contains 400 well-formed prompts across nine intents; D^- contains 500 flawed prompts.

Category	Records
D^+ well-formed prompts	400
Adversarial injection	100
Out-of-scope target/technique	90
Ambiguous / under-specified	30
Contradictory request	64
Fabricated parameter	64
Fabricated CVE	64
Type mismatch (string vs. intent)	64
RBAC violation	24
Total	900

Metrics. Let $y^*(x)$ and $c^*(x)$ denote the gold intent and flaw labels, $\hat{y}(x)$ the recognized intent, and $\hat{c}(x)$ the classified flaw. All three rates are reported as percentages:

$$\mathrm{DAcc} = \frac{1}{|D^+|} \sum_{x \in D^+} \mathbf{1}\big[\hat{y}(x) = y^*(x)\big] \times 100, \tag{3}$$

$$\mathrm{Catch} = \frac{1}{|D^-|} \sum_{x \in D^-} \mathbf{1}\big[\hat{c}(x) = c^*(x)\big] \times 100, \tag{4}$$

$$\mathrm{FP} = \frac{1}{|D^+|} \sum_{x \in D^+} \mathbf{1}\big[\text{a guard flags } x\big] \times 100. \tag{5}$$

DAcc and Catch are the two safety properties a copilot must hold simultaneously — it must do the right thing (DAcc) and must not do the wrong thing (Catch) — while FP measures how often a guard shadow-bans a legitimate operator.

Experiments.

- Exp1 (pipeline value). Compare raw single-shot LLM output against the full guarded pipeline on a 200-prompt stratified holdout (100 D^+, 100 D^-), per backend.
- Exp2 (backend head-to-head). Fix the guardrail stack; compare Gemma vs. Qwen on the full holdout with identical prompts.
- Exp3 (ablation). On the Qwen backend, re-run the full 900-record set once per disabled layer (scope guard off, RBAC off, network validator off, hallucination validator off, validation stack off, self-consistency off, semantic cache off), yielding eight configurations. The "validation stack" here is the entire deterministic validation stack (S5 schema, S6 regex, S6B network, S7 scope) disabled at once; it is a combined ablation, not a single layer, and is reported because it bounds the stack's joint contribution.
- Exp4 (per-class catch). Per-flaw-category catch rate on the Qwen backend.

Implementation notes. The self-consistency gate takes the decoding strategy of Wang et al. [11] from chain-of-thought sampling to classification agreement: it samples k=3 responses $\{z_i\}$ at temperature 0.6 and clusters them against the primary via embedding similarity. The agreement signal is

$$\alpha(x) = \frac{1}{\binom{k}{2}} \sum_{1 \le i < j \le k} \mathbf{1}\big[d_{\cos}\big(\mathbf{e}(z_i), \mathbf{e}(z_j)\big) \le \theta_s\big], \tag{6}$$

where $\mathbf{e}(\cdot)$ is the Sentence-BERT embedding and θ_s the semantic threshold; raw single-sample α=1. With two additional samples, disagreement above θ_s demotes confidence and routes to the hallucination validator, which applies the six-class error taxonomy (impossible CVE, impossible parameter range, unverifiable value, contradictory fields, out-of-scope reference, unresolved type) and selects BLOCK, WARN, or ESCALATE accordingly. The full harness — data, prompts, and evaluation scripts — is versioned with the project source and regenerated by a single command.

VII. IMPLEMENTATION

NeuroShell IRE is implemented as a modular Python service with a FastAPI surface and a clean internal split between the core (S1–S7), the guards (M1–M5), and the llm adapter. Backend swap is a single configuration change; both Ollama models expose the same chat API. The schema validator uses a hand-written recursive validator (kept dependency-light for a cold-start device) rather than a heavyweight runtime. The adapter enforces a per-stage timeout so that an unresponsive model cannot stall the pipeline. Deterministic guards (S5, S6, S6B, S7) and the pre-inference RBAC check run with zero LLM dependency, so the viewer path is a pure function of the request that costs milliseconds.

The test suite comprises 482 automated tests (434 unit, 48 asynchronous integration) measured on 2026-08-26, covering every guard path, RBAC matrix cells, cache hit/miss paths, and the six-class hallucination policy. The dataset and harness are versioned with the source; evaluation is a single reproducible command that regenerates all tables in this paper.

VIII. EVALUATION AND RESULTS

Exp1 — pipeline value. Raw single-shot output made no structured intent available in 100% of flawed cases (catch 0%) and misclassified 22% (Gemma) and 31% (Qwen) of well-formed prompts. Adding the guarded pipeline raised DAcc from 78 to 86% (Gemma) and 69 to 82% (Qwen), raised catch from 0% to 72% (Gemma) and 64% (Qwen), and cut the FP rate from 32% to 14% (Gemma) and from 31% to 15% (Qwen), i.e., the number of well-formed prompts a guard erroneously flags. All FP reductions are conservative well-formed "over-blocks," never a dangerous misclassification toward a privileged intent. The improvement is largest for catch rate because every flawed category is caught by at least one deterministic guard, whereas raw output is never flagged.

Exp2 — backend head-to-head. Under identical guardrails, Gemma leads on DAcc (86 vs. 82) and catch

TABLE II
Exp1: raw LLM output vs. guarded pipeline (200-prompt holdout).

Config	DAcc%	Catch%	FP%
Gemma raw	78	0	32
Gemma guarded	86	72	14
Qwen raw	69	0	31
Qwen guarded	82	64	15

TABLE III
Exp2: identical guardrail stack for both backends (200-prompt holdout).

Backend	DAcc%	Catch%	FP%
Gemma (guarded)	86	72	14
Qwen (guarded)	82	64	15

(72 vs. 64), with similar FP (14 vs. 15). The guarded architecture reduces backend sensitivity: while raw Gemma already outperformed raw Qwen by 9 points, the pipeline closes the DAcc gap to 4 points and holds catch within 8 points, indicating that the deterministic layers dominate residual accuracy for well-formed inputs.

Exp3 — ablation. Table IV shows the effect of removing each layer on the full 900-record set with the Qwen backend. The marginal contribution of a layer ℓ is the difference against the baseline pipeline Π, $\Delta_{\text{catch}}(\ell) = \text{Catch}(\Pi) - \text{Catch}(\Pi \setminus \ell)$, which is the rightmost column. The validation stack as a whole is the dominant safety contributor: disabling it drops catch from 84 to 41 (-43 points) and DAcc from 73.5 to 64.0. Its FP rate also falls sharply, from 34% to 13%, because with S5–S7 disabled only the adversarial detector (M1) and pre-inference RBAC remain able to flag a well-formed prompt — and they do so rarely. Within the stack, the hallucination validator is the single largest catch contributor (-22 points), and the scope guard the largest accuracy contributor (-9.5 points). Removing RBAC increases DAcc by 1.0 point while dropping catch by 20, and cuts the FP rate from 34% to 22%, revealing a benign-versus-safe tension: RBAC refuses legitimate-looking requests that fall outside the caller's role — a deliberate over-block trade accepted for safety. The network validator contributes little at the aggregate level (-5.5 DAcc, -2 catch) because its effect concentrates on the few records whose only detectable flaw is an invalid or policy-violating target. The self-consistency and semantic-cache layers both reduce catch when removed (-4 and -5) while also raising the FP rate ($+11$ and $+13$ points), confirming that the cache improves determinism as well as latency.

Exp4 — per-class catch. On a dedicated 250-record deep-dive set of flawed prompts (100 from D^- plus 150 newly generated, Qwen backend), catch is strongest exactly where deterministic rules exist. Adversarial injection

TABLE IV
Exp3: ablation on the full dataset (Qwen, N=900).

Configuration	DAcc%	Catch%	FP%	ΔCatch
Full pipeline (baseline)	73.5	84	34	—
No scope guard	64.0	73	44	−11
No RBAC guard	74.5	64	22	−20
No network validator	68.0	82	44	−2
No halluc. validator	65.4	62	44	−22
No validation stack	64.0	41	13	−43
No self-consistency	67.5	80	45	−4
No semantic cache	66.0	79	47	−5

is caught at 95.6% (43/45) by M1's lexical/structural layers, and RBAC violations at 100% (12/12) by the deterministic pre-inference role check (a small sample — n=12 — so this is not a tight bound, though the check is rule-based rather than model-sampled). Fabricated parameters (87.5%, 28/32) and type mismatches (81.3%, 26/32) are caught by the S5–S6 syntactic rules; fabricated CVEs (78.1%, 25/32) by the hallucination validator's offline CVE grounding. The residual weak classes are the semantic ones — contradictory requests (59.4%, 19/32), out-of-scope (46.7%, 21/45), and ambiguous (40%, 8/20) — where no deterministic signal exists and the system must rely on the self-consistency gate and the LLM's own agreement. These three classes are also the largest share of the full dataset's flaws: under the 900-record denominators of Table I they sum to 184 of the 500 flawed records (90 out-of-scope, 64 contradictory, 30 ambiguous), versus 45, 32, and 20 in the 250-record deep-dive set above — so per-class reporting against each denominator, not an aggregate catch number, is the appropriate basis for a security claim.

Latency. On CPU-only hardware, end-to-end p50 is ≈16.5 s and p95 ≈70.9 s for cache-miss requests; disabling self-consistency lowers p50 to ≈13.4 s (the two extra samples add roughly 3 s), and the ablation shows it is the largest latency component. Semantic-cache hits resolve in ≈44 ms, a ≈375× speedup, i.e., $\frac{\mathbb{E}[t|\text{miss}]}{\mathbb{E}[t|\text{hit}]} \approx 375$, which makes repeated or scripted intents viable offline. These figures bound the practical constraint behind the design: any guard that adds a third LLM call was rejected because it would push p95 beyond two minutes on this class of hardware.

IX. DISCUSSION

Our results answer the three research questions directly.

RQ1: pipeline value. The guarded pipeline is the difference between unusable and usable output: it turns a hallucinating, injection-prone free-form generator into a structured classifier whose output must pass deterministic checks on the well-formed side, and it catches most flawed inputs before they reach a tool. The catch-rate leap (0→64–72 on the holdout, up to 84 on the 900-record ablation set) is the key result; raw LLM output contains no machine-checkable safety property at all.

RQ2: layer contribution. Hallucination validation and the full validation stack dominate catch; the scope guard dominates accuracy; RBAC trades a point of DAcc for 20 points of catch. Network validation is class-critical but aggregate-invisible without Exp4, which is why per-class reporting is essential for security claims. Crucially, layers interact constructively: removing RBAC damages catch more than removing the network validator, despite RBAC's small DAcc contribution, because the pre-inference fast-fail prevents a class of out-of-role requests from ever being misclassified as authorized.

RQ3: generality. The guardrails hold for both backends with the same ordering of effects; the deterministic stages compress, but do not erase, raw-model quality differences. This supports treating the decoder as a swappable component of a guarded system rather than the security boundary itself.

Threat model boundaries. The design converts a fully open problem — making an LLM trustworthy — into a closed one: classify a single user utterance under known policy. This is defensible because the system retrieves nothing and renders nothing; the standard indirect-injection and tool-injection channels [3], [6] are curtailed at the boundary. What remains is direct adversarial utterance, which M1 and the RBAC matrix address, and measurement shows 95.6% catch on adversarial records.

X. LIMITATIONS AND THREATS TO VALIDITY

Internal validity. Exp3's baseline was measured on the 900-record set while Exp1/Exp2 used a 200-prompt stratified holdout; the DAcc values (73.5 vs. 82) therefore differ across datasets and must not be compared directly. The dataset is synthetic — authored, not sampled from production traffic — so real speech patterns may shift class difficulty. One layer (semantic cache) changes not only latency but also determinism, meaning its ablation conflates a performance effect with a correctness effect; the −5 catch figure may overstate the layer's purely semantic contribution. Because the Exp3 "no validation stack" row disables S5–S7 as one condition, its −43 catch shift is a combined ablation and does not isolate any single stage. RBAC's perfect Exp4 catch rests on n=12 records and reflects a deterministic role lookup, not a general capability. Finally, backend identity (tag, quantization, on-disk size) and the exact Ollama version are pinned in the evaluation manifest but were not independently re-verified against public model cards.

External validity. Experiments use two Ollama-hosted, instruction-tuned 7–9 B models on one laptop. Larger or differently-aligned models may exhibit different failure modes, and cloud models with higher base accuracy may leave less headroom for validation to add. Every experiment is a single run without seed variance or confidence intervals: the pipeline is stochastic (temperature-0.4 decoding, non-deterministic scheduler), so the reported figures are point estimates of bimodal latencies and near-certain class accuracies, not bounds; backend- and seed-averaged reruns with interval reporting are the first planned extension. The hallucination validator's CVE grounding is a hand-curated 20-entry list; identifiers outside it are treated as plausible zero-days rather than auto-blocked, a deliberately conservative policy where a snapshot refreshed from the National Vulnerability Database [15] and scored with EPSS [16] would tighten coverage at the cost of the offline boundary. Construct validity. DAcc and catch are our constructs for "right" and "not wrong"; a user may disagree about borderline classes (contradictory vs. ambiguous), which contributes label noise to the 40–59% classes. Threats unique to offline deployment. We found that offline CPU-only models produce a distinct failure profile — verbose JSON with embedded commentary, and higher per-call variance — that the deterministic stages must absorb; a pipeline tuned on API-grade models would miss these patterns. Finally, RBAC and network policy are evaluations of configuration, so the measured catch reflects the test policy set, not an absolute security guarantee for other operators.

XI. CONCLUSION

We presented NeuroShell IRE, a guarded intent-recognition engine that makes a small, offline, CPU-only LLM reliable enough to gate an operational security copilot under the evaluated threat model. On a 900-record data set spanning eight flaw classes and a per-layer ablation across eight configurations, the pipeline raises catch from 0% to 64–72% on the stratified holdout and 84% on the 900-record ablation set, raises accuracy from 69–78% to 82–86%, and isolates the dominant safety layer (hallucination validation, −22 points of catch if removed). The results hold across two distinct backends, showing that deterministic guardrailing, not model choice, is the load-bearing security property — provided the boundary is closed (no retrieval, no free-form rendering). Residual weakness is concentrated in purely semantic flaw classes, pointing to better need for unsupervised agreement signals and per-operator policy learning as the next steps.

References

[1] G. Deng et al., "PentestGPT: Evaluating and harnessing large language models for automated penetration testing," in Proc. 33rd USENIX Security Symp. (USENIX Security), Philadelphia, PA, USA, 2024, pp. 847–864.

[2] A. Happe and J. Cito, "Getting pwn'd by AI: Penetration testing with large language models," in Proc. 31st ACM Joint Eur. Softw. Eng. Conf. and Symp. on Found. of Softw. Eng. (ESEC/FSE), San Francisco, CA, USA, 2023, pp. 2082–2086.

[3] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in Proc. 16th ACM Workshop Artif. Intell. and Security (AISec), Copenhagen, Denmark, 2023, pp. 79–90.

[4] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Prompt injection attacks and defenses in LLM-integrated applications," arXiv:2310.12815, 2023.

[5] D. Kang, X. Li, I. Stoica, C. Guestrin, M. Zaharia, and T. Hashimoto, "Exploiting programmatic behavior of LLMs: Dual-use through standard security attacks," in Proc. IEEE Security and Privacy Workshops (SPW), San Francisco, CA, USA, 2024, pp. 132–143.
[6] OWASP Foundation, "OWASP Top 10 for Large Language Model Applications, 2025," genai.owasp.org, 2025. [Online]. Available: https://genai.owasp.org/
[7] Z. Ji et al., "Survey of hallucination in natural language generation," ACM Comput. Surv., vol. 55, no. 12, pp. 1–38, 2023.
[8] P. Manakul, A. Liusie, and M. J. F. Gales, "SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models," in Proc. EMNLP, Singapore, 2023, pp. 9004–9017.
[9] L. Kuhn, Y. Gal, and S. Farquhar, "Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation," in Proc. ICLR, Kigali, Rwanda, 2023.
[10] S. Farquhar, J. Kossen, L. Kuhn, and Y. Gal, "Detecting hallucinations in large language models using semantic entropy," Nature, vol. 630, no. 8017, pp. 625–630, 2024.
[11] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. H. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-consistency improves chain of thought reasoning in language models," in Proc. ICLR, Kigali, Rwanda, 2023.
[12] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Proc. NeurIPS, Vancouver, Canada, 2020, pp. 9459–9474.
[13] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proc. EMNLP-IJCNLP, Hong Kong, 2019, pp. 3982–3992.
[14] Y. Rekhter, B. Moskowitz, D. Karrenberg, G. de Groot, and E. Lear, "Address allocation for private internets," IETF, RFC 1918, 1996. [Online]. Available: https://www.rfc-editor.org/rfc/rfc1918
[15] National Institute of Standards and Technology, "National Vulnerability Database (NVD)," nvd.nist.gov. [Online]. Available: https://nvd.nist.gov/
[16] FIRST.org, "Exploit Prediction Scoring System (EPSS)," first.org. [Online]. Available: https://www.first.org/epss/
[17] G. F. Lyon, Nmap Network Scanning. Sunnyvale, CA, USA: Insecure.Com LLC, 2009.
[18] Rapid7, "Metasploit Framework," rapid7.com. [Online]. Available: https://www.metasploit.com/